

점프 투 자바

박응용 지음(위키독스 운영자)

이지스퍼블리싱

03

자바의 기초 - 자료형



03-5 StringBuffer

03-6 배열

03-7 리스트

03-5 StringBuffer

- 문자열을 추가하거나 변경할 때 주로 사용하는 자료형
- **append**
  - StringBuffer 객체를 생성하고 문자열을 생성하는 예제

Do it! 코딩해 보세요

```
StringBuffer sb = new StringBuffer();  // StringBuffer 객체 sb 생성
sb.append("hello");
sb.append(" ");
sb.append("jump to java");
String result = sb.toString();
System.out.println(result);  // "hello jump to java" 출력
```

실행 결과

```
hello jump to java
```

  - StringBuffer 객체를 한 번만 생성

03-5 StringBuffer

- 문자열을 추가하거나 변경할 때 주로 사용하는 자료형
- **append**
  - StringBuffer 대신 String 자료형을 사용하여 문자열을 생성하는 예제

Do it! 코딩해 보세요

```
String result = "";
result += "hello";
result += " ";
result += "jump to java";
System.out.println(result);  // "hello jump to java" 출력
```

실행 결과

```
hello jump to java
```

  - String 자료형에 + 연산이 있을 때마다 새로운 String 객체를 생성 – 총 4개의 String 자료형 객체

03-5 StringBuffer

- 문자열을 추가하거나 변경할 때 주로 사용하는 자료형
- **append**
  - String 자료형은 값이 한 번 생성되면 변경할 수 없음
  - StringBuffer 자료형은 값을 변경할 수 있으므로 생성된 값을 언제든지 수정할 수 있음

➡ 문자열을 추가하거나 변경하는 작업이 많으면 StringBuffer를,
문자열을 추가하거나 변경하는 작업이 적으면 String을 사용하는 것이 유리

03-5 StringBuffer

insert

- 특정 위치에 원하는 문자열을 삽입

Do it! 코딩해 보세요

```
StringBuffer sb = new StringBuffer();
sb.append("jump to java");
sb.insert(0, "hello ");
System.out.println(sb.toString());
```

실행 결과

```
hello jump to java
```

03-5 StringBuffer

substring

- String 자료형의 substring 메서드와 동일하게 동작
- substring(시작 위치, 끝 위치)와 같이 사용하면
  StringBuffer 객체의 시작 위치에서 끝 위치까지의 문자를 뽑아냄

Do it! 코딩해 보세요

```
StringBuffer sb = new StringBuffer();
sb.append("Hello jump to java");
System.out.println(sb.substring(0, 4));
```

실행 결과

```
Hell
```

03-6 배열

- **배열(array)**
  - 자료형의 종류가 아니라 자료형의 집합을 의미
    - int 배열로 1부터 10까지 숫자 중 홀수의 집합을 표현

```
int[] odds = {1, 3, 5, 7, 9};
```

    - String 배열로 요일의 집합을 표현

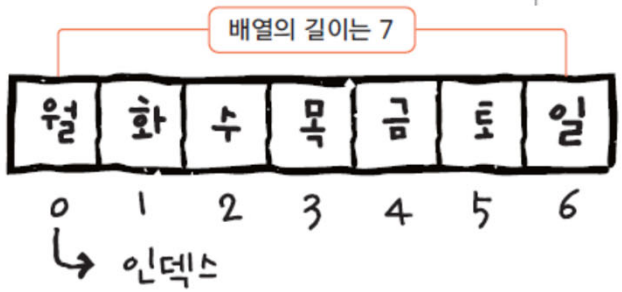
```
String[] weeks = {"월", "화", "수", "목", "금", "토", "일"};
```

03-6 배열

▪ 배열의 길이 설정하기

- 배열의 길이를 먼저 설정한 뒤, 배열 변수를 생성하고 그 값을 나중에 대입하는 방법

```
String[] weeks = new String[7];
weeks[0] = "월";
weeks[1] = "화";
weeks[2] = "수";
weeks[3] = "목";
weeks[4] = "금";
weeks[5] = "토";
weeks[6] = "일";
```



- 초깃값 없이 배열 변수를 만들 때 숫잣값을 넣어 길이를 정해 주지 않으면 컴파일 오류 발생

03-6 배열

▪ 배열값에 접근하기

- 배열에서 특정 값을 얻으려면 인덱스를 이용

Do it! 코딩해 보세요

```
String[] weeks = {"월", "화", "수", "목", "금", "토", "일"};
System.out.println(weeks[3]);
```

실행 결과

```
목
```

03-6 배열

▪ 배열의 길이 구하기

- for 문과 함께 사용해 length를 사용하여 배열의 길이를 구함

Do it! 코딩해 보세요

```
String[] weeks = {"월", "화", "수", "목", "금", "토", "일"};
for (int i = 0; i < weeks.length; i++) {
    System.out.println(weeks[i]);
}
```

실행 결과

```
월
화
수
목
금
토
일
```

03-6 배열

배열 오류

- 자바 코드에서 가장 흔한 오류

```
System.out.println(weeks[7]);  // 여덟 번째 배열값이 없으므로 ArrayIndexOutOfBoundsException
오류 발생
```

  - 요일의 배열 길이는 7인데 여덟 번째 값을 얻으려면 ArrayIndexOutOfBoundsException 오류 발생

03-7 리스트

- **리스트(list)**
  - 배열과 비슷하지만 편리한 기능이 더 많음
  - 배열은 크기가 정해져 있지만 리스트는 변한다는 것이 가장 큰 차이점
    - 예를 들어 배열의 크기를 10개로 정했다면 10개를 초과하는 값을 담을 수 없지만 리스트는 크기가 정해져 있지 않아 원하는 만큼 값을 담을 수 있음



03-7 리스트

ArrayList

add

- ArrayList의 add 메서드를 사용하면 요솟값을 추가할 수 있음

Do it! 코딩해 보세요

```
import java.util.ArrayList;

public class Sample {
    public static void main(String[] args) {
        ArrayList pitches = new ArrayList();
        pitches.add("138");
        pitches.add("129");
        pitches.add("142");
    }
}
```



- ArrayList의 객체인 pitches에 add 메서드를 이용해 투구 속도를 저장

03-7 리스트

ArrayList

get

- ArrayList의 get 메서드를 사용하면 특정 인덱스의 값을 추출할 수 있음

Do it! 코딩해 보세요

```
import java.util.ArrayList;

public class Sample {
    public static void main(String[] args) {
        ArrayList pitches = new ArrayList();
        pitches.add("138");
        pitches.add("129");
        pitches.add("142");
        System.out.println(pitches.get(1));
    }
}
```

실행 결과

```
129
```

03-7 리스트

ArrayList

size

- size 메소드는 ArrayList의 요소의 개수를 리턴

Do it! 코딩해 보세요

```
(... 생략 ...)
System.out.println(pitches.size());
```

실행 결과

```
3
```

contains

- contains 메서드는 리스트 안에 해당 항목이 있는지 판별해 그 결과를 boolean으로 리턴

Do it! 코딩해 보세요

```
(... 생략 ...)
System.out.println(pitches.contains("142"));
```

실행 결과

```
true
```

03-7 리스트

ArrayList

remove

- remove(객체) 방식

Do it! 코딩해 보세요

```
(... 생략 ...)
System.out.println(pitches.remove("129"));
```

129를 리스트에서 삭제하고, true를 리턴한다.

실행 결과

```
true
```

- remove(인덱스) 방식

Do it! 코딩해 보세요

```
(... 생략 ...)
System.out.println(pitches.remove(0));
```

pitches의 첫 번째 항목이 138이므로, 138을 삭제한 뒤 138을 리턴한다.

실행 결과

```
138
```

03-7 리스트

제네릭스(generics)

- 자료형을 안전하게 사용할 수 있도록 만들어 주는 기능
- 제네릭스를 사용하면 자료형을 강제로 바꿀 때 생길 수 있는 캐스팅 오류를 줄일 수 있음

일반적인 방식	ArrayList<String> pitches = new ArrayList<String>();
선호되는 방식	ArrayList<String> pitches = new ArrayList<>();

이 부분의 자료형은 명확하므로 굳이 적지 않아도 된다.

- 제네릭스 도입 전후의 차이는 ArrayList 자료형 다음에 <String>이 있는가의 여부
  - 제네릭스를 표현한 <String>은 'ArrayList에 담을 수 있는 자료형은 String뿐이다'라는 뜻

03-7 리스트

▪ 제네릭스(generics)

▪ 제네릭스 사용 전후의 차이 예

Do it! 코딩해 보세요

```
ArrayList pitches = new ArrayList();
pitches.add("138");
pitches.add("129");

String one = (String) pitches.get(0);
String two = (String) pitches.get(1);
```



Do it! 코딩해 보세요

```
ArrayList<String> pitches = new ArrayList<>();
pitches.add("138");
pitches.add("129");

String one = pitches.get(0);  // 형 변환이 필요 없다.
String two = pitches.get(1);  // 형 변환이 필요 없다.
```

- 제네릭스로 자료형을 선언만 하면 그 이후로는 자료형을 형 변환하는 과정이 필요 없음
- 형 변환과 관련된 불필요한 코딩을 줄일 수 있고, 잘못된 형 변환 때문에 발생하는 런타임 오류를 방지

03-7 리스트

▪ 다양한 방법으로 ArrayList 만들기

1. ArrayList의 add 메서드를 사용해 ArrayList 객체에 요소 추가

Do it! 코딩해 보세요

```
import java.util.ArrayList;

public class Sample {
    public static void main(String[] args) {
        ArrayList<String> pitches = new ArrayList<>();  // 제네릭스를 사용한 표현
        pitches.add("138");
        pitches.add("129");
        pitches.add("142");
        System.out.println(pitches);  // [138, 129, 142] 출력
    }
}
```

실행 결과

```
[138, 129, 142]
```

03-7 리스트

▪ 다양한 방법으로 ArrayList 만들기

2. java.util.Arrays 클래스의 asList 메서드를 사용해 문자열 배열로 생성

Do it! 코딩해 보세요

```
import java.util.ArrayList;
import java.util.Arrays;

public class Sample {
    public static void main(String[] args) {
        String[] data = {"138", "129", "142"};  // 이미 투구 수 데이터 배열이 있다.
        ArrayList<String> pitches = new ArrayList<>(Arrays.asList(data));
        System.out.println(pitches);  // [138, 129, 142] 출력
    }
}
```

실행 결과

```
[138, 129, 142]
```

03-7 리스트

▪ 다양한 방법으로 ArrayList 만들기

3. String 배열 대신 String 자료형을 여러 개 전달하여 생성

Do it! 코딩해 보세요

```
import java.util.ArrayList;
import java.util.Arrays;

public class Sample {
    public static void main(String[] args) {
        ArrayList<String> pitches = new ArrayList<>(Arrays.asList("138", "129", "142"));
        System.out.println(pitches);
    }
}
```

실행 결과

```
[138, 129, 142]
```

03-7 리스트

String.join

- 138, 129, 142 세 요소로 이루어진 ArrayList의 각 요소를 콤마(,)로 구분해서 1개의 문자열로 만들기 → String.join 사용

Do it! 코딩해 보세요

```
import java.util.ArrayList;
import java.util.Arrays;

public class Sample {
    public static void main(String[] args) {
        ArrayList<String> pitches = new ArrayList<>(Arrays.asList("138", "129", "142"));
        String result = String.join(",", pitches);
        System.out.println(result);  // "138,129,142" 출력
    }
}
```

실행 결과

```
138, 129, 142
```

03-7 리스트

▪ 리스트 정렬하기

- 138, 129, 142 세 요소로 이루어진 ArrayList를 순서대로 정렬 → sort 메서드 사용

Do it! 코딩해 보세요

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;

public class Sample {
    public static void main(String[] args) {
        ArrayList<String> pitches = new ArrayList<>(Arrays.asList("138", "129", "142"));
        pitches.sort(Comparator.naturalOrder());  // 오름차순으로 정렬
        System.out.println(pitches);  // [129, 138, 142] 출력
    }
}
```

실행 결과

```
[129, 138, 142]
```

- 오름차순(순방향) 정렬: Comparator.naturalOrder()
- 내림차순(역방향) 정렬: Comparator.reverseOrder()



감사합니다.